



Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ФАКУЛЬТЕТ «Программной инженерии и компьютерной техники»

ОБРАЗОВАТЕЛЬНАЯ ПРОГРАММА «Проектирование и разработка систем больших данных»

НАПРАВЛЕНИЕ ПОДГОТОВКИ (СПЕЦИАЛЬНОСТЬ) «09.04.04 Программная инженерия»

Программной инженерии и компьютерной техники (ФПИ и КТ)

ОТЧЕТ

по лабораторной работе №1

по курсу «Структуры и алгоритмы в базах данных и распределенных
системах»

на тему: «Реализация алгоритмов хэширования (perfect, extendible, min)»

Обучающийся: Постнов С. А., Р4135
(Ф.И.О, № группы)

Преподаватель Платонов А. В., доцент
(Ф.И.О, должность)

Преподаватель Портнов П. В., ассистент
(Ф.И.О, должность)

2026 г.

СОДЕРЖАНИЕ

1	Теоретическая часть	3
1.1	Perfect hashing	3
1.2	Extendible hashing	3
1.3	Min hash	4
2	Практическая часть	5
2.1	Реализация perfect hash таблицы	5
2.2	Реализация extendible hash таблицы	5
2.3	Реализация min hash таблицы LSH индекса	6
3	Исследовательская часть	7
3.1	Аппаратные характеристики	7
3.2	Методика замеров perfect hashing	7
3.3	Методика замеров extendible hashing	7
3.4	Методика замеров min hash	7
3.5	Результаты для perfect hashing	8
3.6	Результаты для extendible hashing	10
3.7	Результаты для min hash	13
4	Вывод	15

1 Теоретическая часть

1.1 Perfect hashing

Perfect hash для множества ключей S — это хэш-функция h , которая отображает ключи из S в индексы таблицы без коллизий. В классическом варианте (FKS) используется двухуровневая схема: первичная таблица распределяет ключи по корзинам, а для каждой корзины строится вторичная таблица, в которой подбираются параметры так, чтобы коллизий не было. Основными идеями алгоритма являются:

- первичная таблица имеет размер, равный ближайшему простому $\geq n$;
- в ячейке хранится либо одна пара **key/value**, либо указатель на вторичную таблицу;
- при коллизии создаётся/перестраивается вторичная таблица: увеличивается её размер и перебираются различные хэш-функции до получения раскладки без коллизий.

1.2 Extendible hashing

Extendible hashing предназначен для динамических наборов данных, где число ключей заранее неизвестно и структура должна расти и уменьшаться без полной перестройки таблицы. В отличие от *perfect hashing*, здесь допускаются коллизии, но они локализуются в бакетах фиксированной ёмкости. Основные элементы алгоритма:

- директория содержит 2^d указателей на бакеты, где d — глобальная глубина;
- каждый бакет имеет локальную глубину, определяющую, по скольким младшим битам хэша он адресуется;
- при переполнении бакет расщепляется, а при необходимости директория удваивается;
- при удалении возможны обратные операции: слияние соседних бакетов и уменьшение директории.

Такая схема обеспечивает амортизированно быстрые операции `insert/get/update/delete` и хорошо подходит для внешней памяти, так как бакеты можно размещать непосредственно в файле и адресовывать через отображение файла в память.

1.3 Min hash

Min hash предназначен для эффективного поиска похожих объектов в больших коллекциях, используя хэширование, чувствительное к локальности. Мера сходства — коэффициент Жаккара $J(A, B) = |A \cap B| / |A \cup B|$, где A и B — множества *шинглов* (подстрок длины k , извлечённых скользящим окном) двух документов. *Min hash* даёт компактную аппроксимацию Жаккара, то есть для набора хэш-функций $h_1, \dots, h_m$ сигнатура документа — это вектор минимумов $\sigma_i = \min_{s \in A} h_i(s)$. Вероятность совпадения $\sigma_i(A) = \sigma_i(B)$ равна $J(A, B)$, поэтому долю совпавших компонент можно использовать как оценку сходства.

Locality-Sensitive Hashing (LSH) ускоряет поиск кандидатов — сигнатура делится на b полос по r элементов каждая. Два документа попадают в один бакет хотя бы одной полосы с вероятностью $1 - (1 - s^r)^b$, где s — истинный Жаккар. Порог чувствительности $s^* \approx (1/b)^{1/r}$ задаётся выбором b и r . Полосовая схема позволяет за линейное время построить список пар-кандидатов, а затем верифицировать их точным сравнением шинглов.

2 Практическая часть

2.1 Реализация perfect hash таблицы

Публичный интерфейс реализован в файлах `inc/perfect.h` и `src/perfect.c`:

- `create_perfect_table` создаёт таблицу нужного размера;
- `perfect_table_from_csv` строит таблицу по входному CSV;
- `perfect_get` выполняет поиск значения по ключу;
- `free_perfect_table` освобождает память.

Структура `perfect_hash_table_t` содержит массив указателей на `ceil_value_t`. Каждая ячейка либо хранит одну пару `key/value`, либо указывает на вторичную таблицу (вложенный `perfect_hash_table_t`). Для хэширования строк используются функции из `inc/hash.h`. При перестроении вторичных таблиц перебирается набор хэш-функций до построения раскладки без коллизий.

2.2 Реализация extendible hash таблицы

Основной код расположен в файле `extendible/extendible.go`. Таблица хранится не в оперативной памяти как отдельная структура с последующей сериализацией, а непосредственно в файле, отображённом в память с помощью `mmap`. Файл разбит на три логические области:

- 1) заголовок с метаданными таблицы (магическое число, версия формата, глобальная глубина, размер бакета);
- 2) директория фиксированного максимального размера, содержащая идентификаторы бакетов;
- 3) область бакетов, где для каждого бакета хранятся локальная глубина, число записей и массив пар `key/value/hash`.

Публичный интерфейс включает операции `Insert`, `Update`, `Delete`, `Get`, а также открытие и закрытие таблицы в файле. При переполнении вызывается `split` бакета, при удалении — `merge buddy`-бакетов и, если возможно, `shrink` директории.

2.3 Реализация min hash таблицы LSH индекса

Основные файлы: `min/minhash.go` (ядро алгоритма) и `min/benchmark_test.go` (генерация случайного корпуса для замеров).
Архитектура индекса:

- 1) из текста документа извлекаются шинглы функцией `shingleHashes` скользящим окном длины $k = 2$ слова; значение каждого шингла — FNV-1a хэш строки;
- 2) функцией `signatureFor` строится min hash-сигнатура из $m = 64$ хэш-функций на основе `splitmix64`;
- 3) при построении индекса (`Build`) сигнатуры всех документов разбиваются на $b = 8$ полос по $r = 8$ компонент; каждая полоса хэшируется FNV-полиномом в ключ бакета (`bandKey`);
- 4) при добавлении нового документа (`Add`) его бакеты объединяются с существующими;
- 5) поиск дубликатов (`FindDuplicates`) извлекает кандидатов из бакетов и проверяет пары точным подсчётом Жаккара на сохранённых шинглах;
- 6) `FullScanDuplicates` является базовым алгоритмом сравнения: каждый документ проверяется против всех остальных за $O(n^2)$.

Параметры m , b , r , k передаются через структуру `Config`.

3 Исследовательская часть

3.1 Аппаратные характеристики

Все замеры проводились на ноутбуке со следующей конфигурацией:

- операционная система: macOS 26.3.1 (arm64);
- процессор: Apple M3 Pro, 11 ядер;
- оперативная память: 18 ГБ;
- версия Go (для extendible и min): go1.25, версия C (для perfect): c17;

Во время замеров не выполнялись ресурсоёмкие фоновые задачи, а ноутбук был подключён к сети электропитания для обеспечения максимальной производительности.

3.2 Методика замеров perfect hashing

Для замеров использована программа `src/benchmark.c`. Для каждого датасета из `data/bench/manifest.txt`:

- 1) создаётся perfect hash таблица из csv датасета;
- 2) измеряется время чтения всех n ключей (`perfect_get`).

Каждый прогон усредняется по `BENCH_ITERS` (по умолчанию 100).

3.3 Методика замеров extendible hashing

Для extendible hashing использованы встроенные benchmark-тесты Go и `pprof`. Измерялись четыре операции: `insert`, `update`, `delete`, `get`. Для каждой мощности набора данных выполнялись бенчмарки со считыванием времени на операцию, пропускной способности, количества аллокаций и объёма выделенной памяти.

3.4 Методика замеров min hash

Для min hash использованы встроенные benchmark-тесты Go. Измерялись три операции:

- **build** — построение индекса из корпуса размером N ;
- **add** — добавление N документов по одному в уже существующий индекс;
- **fullscan** — полный обход: для каждого из N документов запускается линейный поиск дубликатов по всему корпусу.

Для каждой мощности набора данных фиксировались время на элемент, пропускная способность, количество итераций и 95% доверительный интервал.

3.5 Результаты для perfect hashing

Таблица 3.1 содержит усреднённые значения задержки поиска и пропускной способности.

Таблица 3.1 – Результаты бенчмарка perfect hash (среднее по итерациям)

N	поиск, нс/оп	поиск, млн оп/с
20000	23.1 ± 0.7	44.19
50000	22.1 ± 0.1	45.33
100000	22.2 ± 0.1	45.07
250000	24.6 ± 0.5	40.94
500000	28.0 ± 0.7	36.08
750000	26.7 ± 0.7	37.77
1100000	29.9 ± 0.7	33.63
2000000	30.5 ± 0.7	33.06
5000000	34.3 ± 0.9	29.36
7500000	36.1 ± 1.0	27.94

Графики результатов замеров представлены на рисунках 3.1 и 3.2:

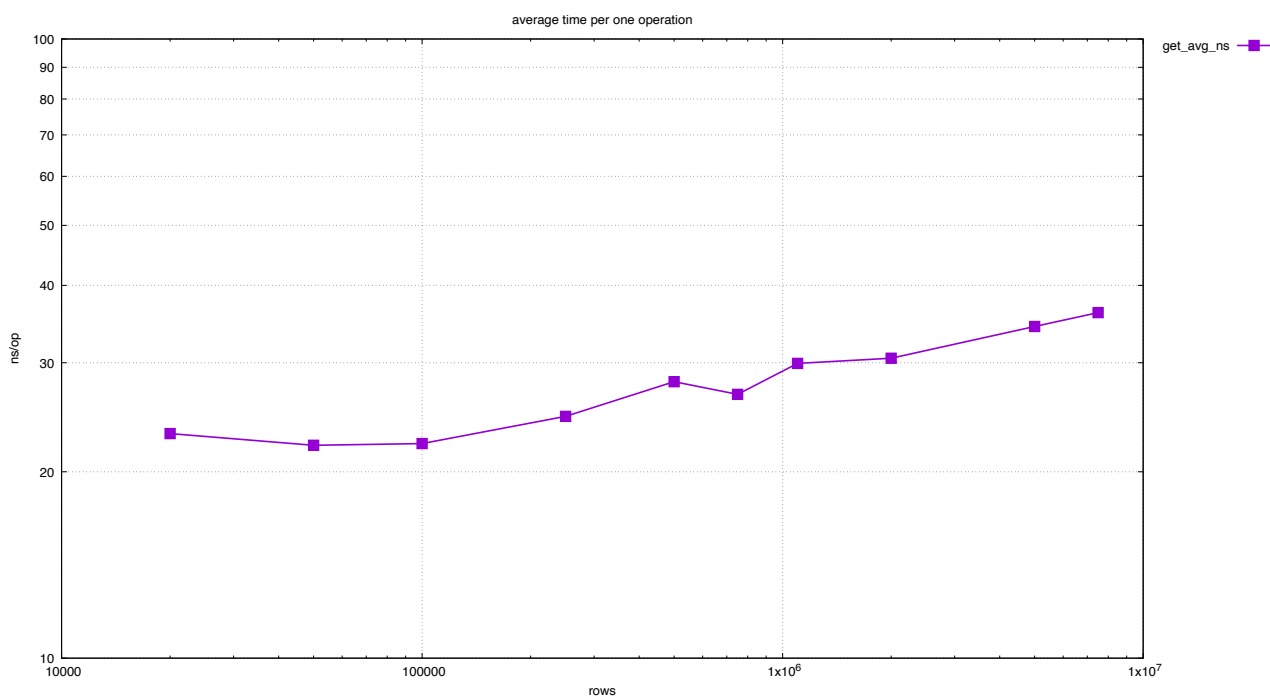


Рисунок 3.1 – Средняя задержка операции поиска для perfect hash

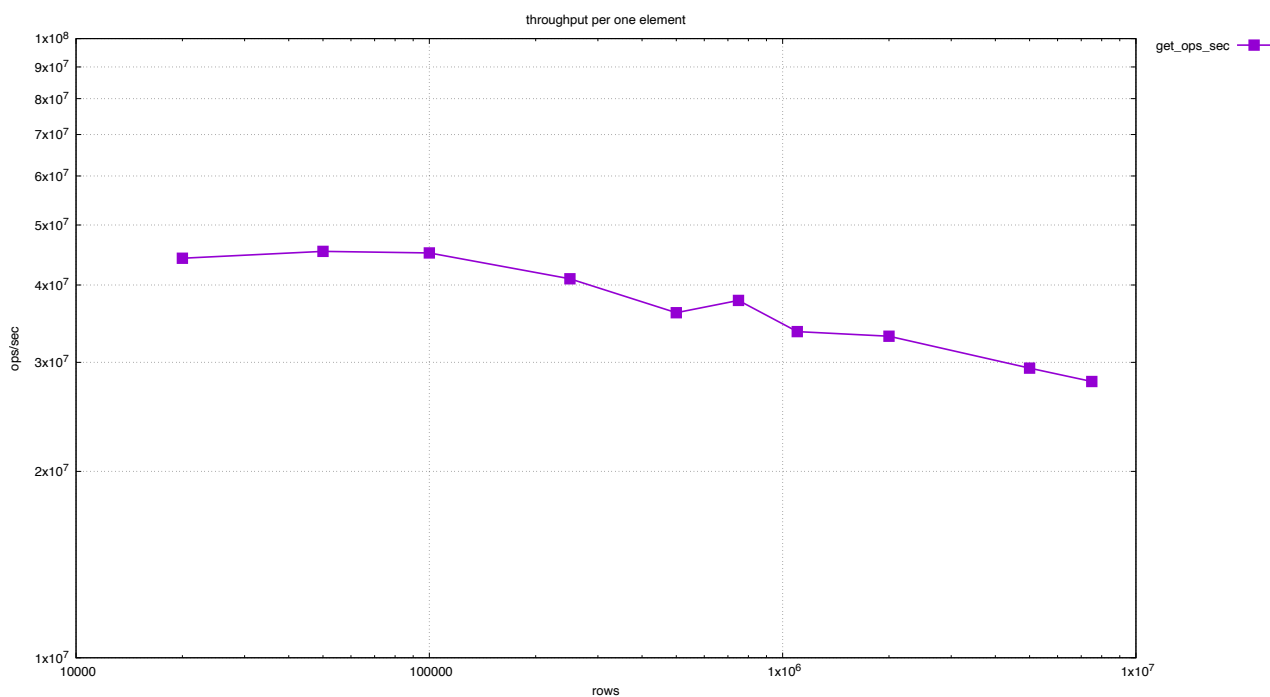


Рисунок 3.2 – Пропускная способность операции поиска для perfect hash

По таблице 3.1 и графикам можно сделать вывод, что при росте N задержка `get` слабо растет, примерно с 22–23 нс/оп на малых наборах до 34–36 нс/оп на максимальных. Пропускная способность, соответственно, снижается с порядка 45 до 28 млн оп/с, что ожидаемо из-за роста рабочей области и нагрузки на кэш. В целом для статического множества ключей perfect

hashing сохраняет высокую производительность поиска и предсказуемое время доступа.

3.6 Результаты для extendible hashing

Таблицы 3.2, 3.3 и 3.4 содержат результаты бенчмарка extendible hashing для всех измеренных размеров набора данных.

Таблица 3.2 – Результаты бенчмарка задержки в extendible hashing (среднее по итерациям)

N	итерации	вставка, нс/оп	обновление, нс/оп	удаление, нс/оп	поиск, нс/оп
1000	27894	43.42 ± 0.035	32.61 ± 0.019	40.86 ± 0.024	32.41 ± 0.020
5000	5374	44.61 ± 0.060	25.88 ± 0.023	35.37 ± 0.032	25.16 ± 0.023
10000	2610	45.85 ± 0.072	27.47 ± 0.028	40.36 ± 0.048	26.73 ± 0.028
50000	483	49.59 ± 0.109	30.86 ± 0.053	70.55 ± 0.355	30.81 ± 0.046
100000	208	57.37 ± 0.133	32.03 ± 0.063	94.91 ± 0.861	31.87 ± 0.051
500000	19	123.7 ± 0.699	36.31 ± 0.103	310.8 ± 6.781	35.95 ± 0.087
1000000	10	214.1 ± 2.413	54.02 ± 0.440	559.7 ± 19.71	53.44 ± 0.416

Таблица 3.3 – Результаты бенчмарка пропускной способности в extendible hashing (среднее по итерациям)

N	итерации	вставка, млн оп/с	обновление, млн оп/с	удаление, млн оп/с	поиск, млн оп/с
1000	27894	23.0 ± 0.018	30.7 ± 0.018	24.5 ± 0.015	30.9 ± 0.019
5000	5374	22.4 ± 0.030	38.6 ± 0.021	28.3 ± 0.025	39.7 ± 0.036
10000	2610	21.8 ± 0.034	36.4 ± 0.029	24.8 ± 0.047	37.4 ± 0.035
50000	483	20.2 ± 0.044	32.4 ± 0.053	14.2 ± 0.071	32.5 ± 0.047
100000	208	17.4 ± 0.051	31.2 ± 0.061	10.5 ± 0.097	31.4 ± 0.051
500000	19	8.1 ± 0.069	27.5 ± 0.098	3.2 ± 0.035	27.8 ± 0.083
1000000	10	4.7 ± 0.048	18.5 ± 0.151	1.8 ± 0.063	18.7 ± 0.146

Таблица 3.4 – Результаты замера времени прогрева (warmup) в extendible hashing

N	итерации	время warmup, нс/оп
1000	1681	815416 ± 128.5
5000	8356	1093872 ± 72.15
10000	1124	1090091 ± 196.1
50000	1166	1062745 ± 170.0
100000	283	3677399 ± 588.4
500000	250	6045153 ± 653.3
1000000	100	10683703 ± 1008

Из таблиц можно сделать вывод, что операции **get** и **update** наиболее стабильны по мере роста набора данных. Операция **insert** заметно дороже из-за split бакетов и расширения директории, а **delete** оказывается самой тяжёлой на больших N из-за merge/shrink после удаления элементов. Дополнительные профилировочные замеры показывают, что основной объём аллокаций приходится на **insert**; для **update**, **delete** и **get** объём дополнительных аллокаций почти не растёт и определяется в основном накладными расходами рантайма.

Графики результатов замеров extendible hashing представлены на рисунках ниже.

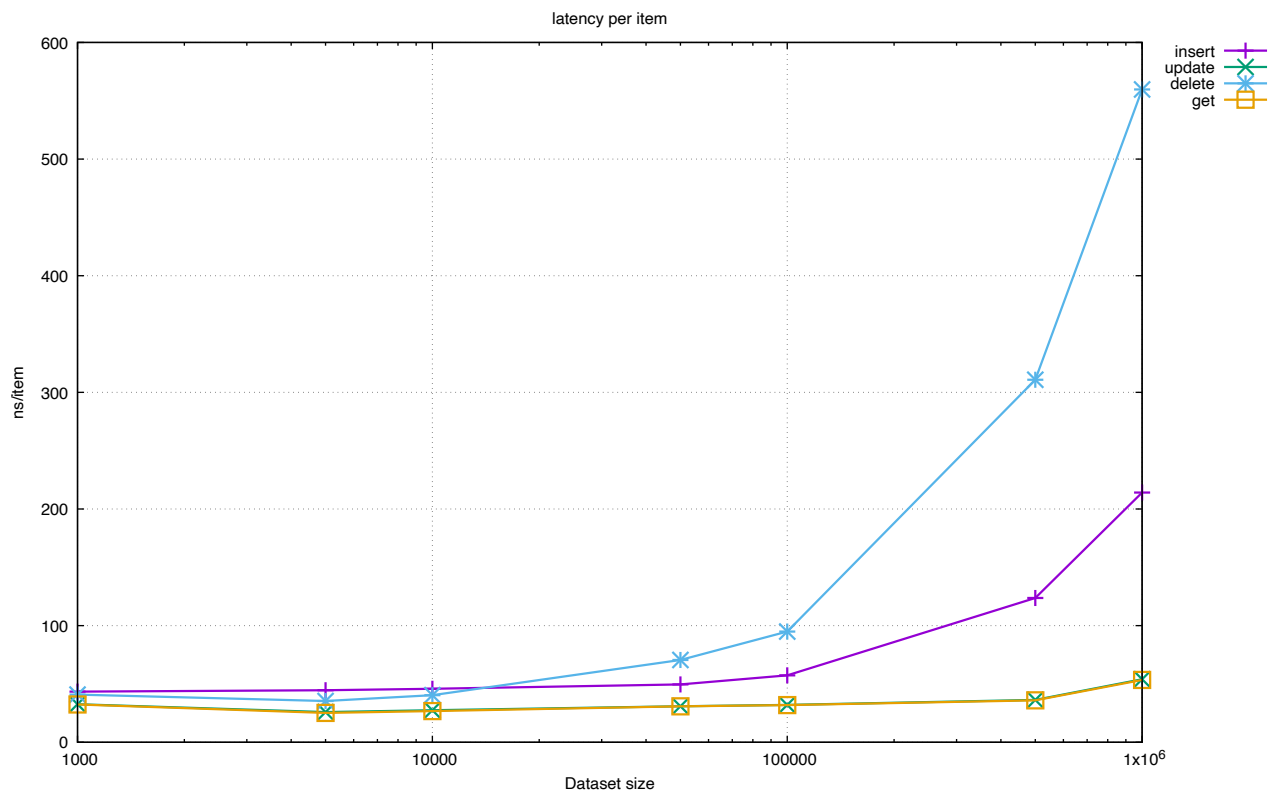


Рисунок 3.3 – Задержка операций extendible hashing на элемент

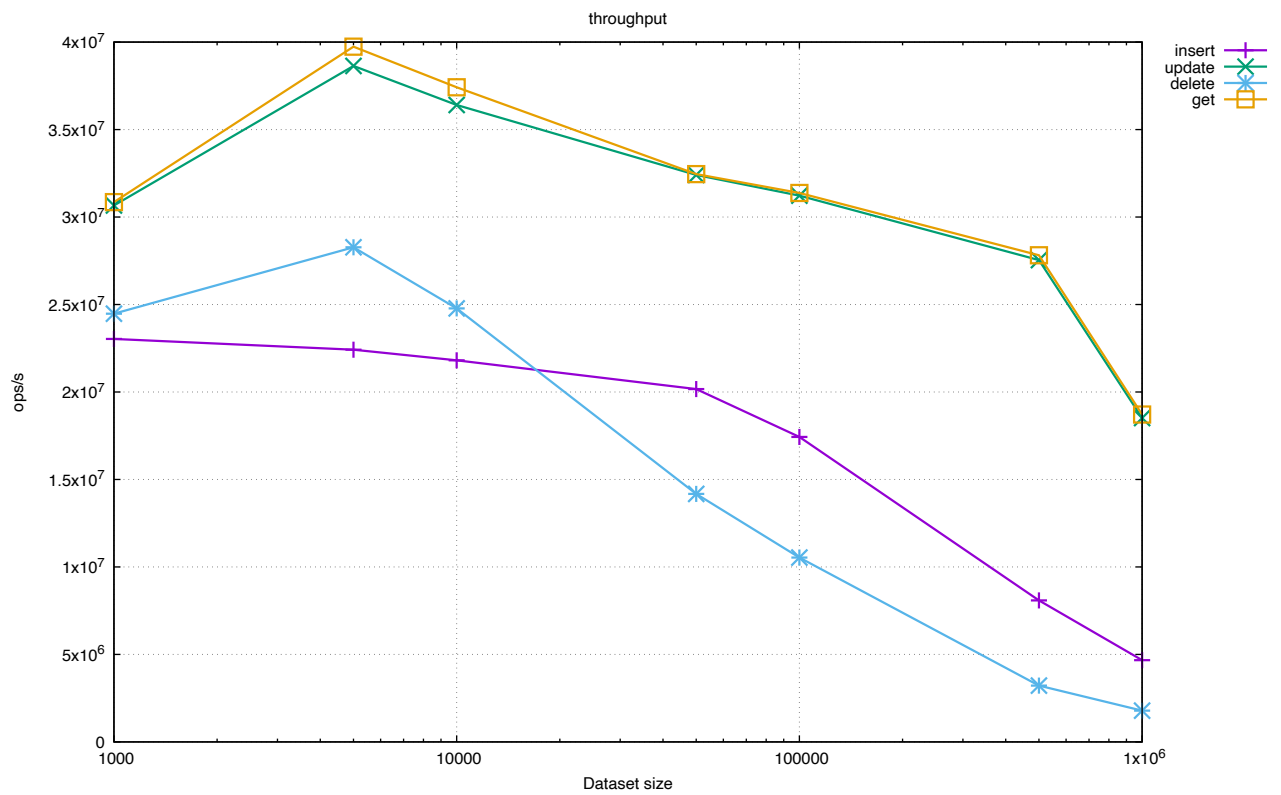


Рисунок 3.4 – Пропускная способность операций extendible hashing

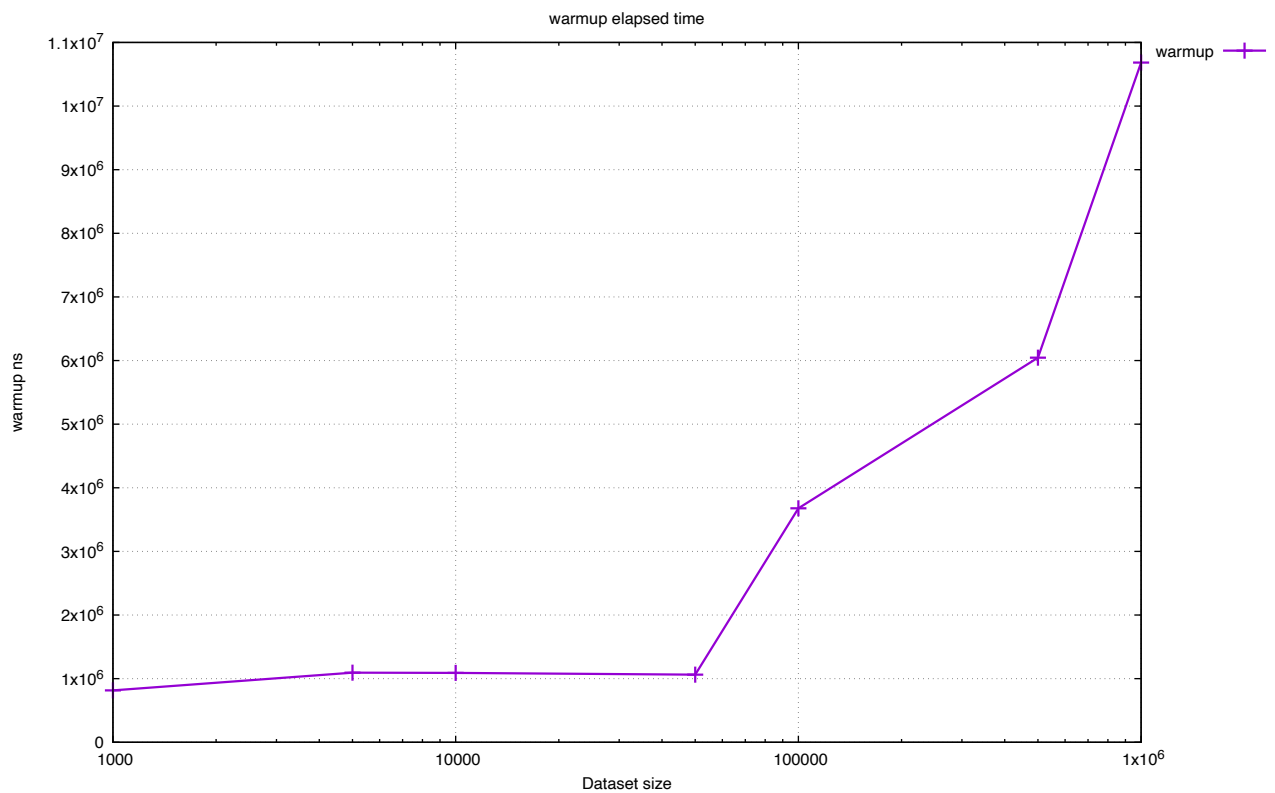


Рисунок 3.5 – Время прогрева (warmup) для extendible hashing

3.7 Результаты для min hash

Таблица 3.5 содержит задержку операций, таблица 3.6 — пропускную способность.

Таблица 3.5 – Результаты бенчмарка задержки в min hash (среднее по итерациям)

N	построение, тыс нс/элемент	добавление, тыс нс/элемент	полный обход, тыс нс/элемент
500	14.940 ± 0.008846 ($n = 9280$)	15.329 ± 0.004471 ($n = 9378$)	182.112 ± 0.1235 ($n = 789$)
1000	15.164 ± 0.006001 ($n = 4753$)	15.238 ± 0.006451 ($n = 4718$)	364.406 ± 0.3443 ($n = 196$)
2500	15.149 ± 0.009802 ($n = 1902$)	15.258 ± 0.01479 ($n = 1891$)	911.679 ± 1.548 ($n = 31$)
5000	15.145 ± 0.01144 ($n = 950$)	15.338 ± 0.01724 ($n = 938$)	1825.010 ± 2.314 ($n = 7$)
7500	15.242 ± 0.01519 ($n = 628$)	15.453 ± 0.01588 ($n = 621$)	2750.945 ± 3.632 ($n = 3$)

Таблица 3.6 – Результаты бенчмарка пропускной способности в min hash (среднее по итерациям)

N	построение, оп/с	добавление, оп/с	полный обход, оп/с
500	66936 ± 40 ($n = 9280$)	65234 ± 19 ($n = 9378$)	5491 ± 4 ($n = 789$)
1000	65944 ± 26 ($n = 4753$)	65624 ± 28 ($n = 4718$)	2744 ± 3 ($n = 196$)
2500	66012 ± 43 ($n = 1902$)	65538 ± 64 ($n = 1891$)	1097 ± 2 ($n = 31$)
5000	66029 ± 50 ($n = 950$)	65196 ± 73 ($n = 938$)	548 ± 1 ($n = 7$)
7500	65610 ± 65 ($n = 628$)	64711 ± 66 ($n = 621$)	364 ± 1 ($n = 3$)

Операции **build** и **add** демонстрируют практически постоянную задержку (около 15 тыс нс/элемент) при росте корпуса, что объясняется независимой обработкой каждого документа при вычислении сигнатуры. Задержка **fullscan** растёт квадратично ($O(n^2)$), поскольку для каждого из N документов выполняется линейный перебор остальных документов корпуса.

4 Вывод

Реализована perfect hash таблица для строковых ключей в статическом сценарии (построение по CSV и поиск), extendible hash таблица с файловым хранением на базе `mmap`, поддерживающая динамические операции вставки, чтения, обновления и удаления, а также min hash LSH индекс для поиска почти дубликатов в текстовых коллекциях.

Проведены замеры производительности на наборах данных разного размера. Для perfect hashing подтверждена эффективность на статическом множестве ключей: операция `get` показывает низкую задержку и высокую пропускную способность на всем диапазоне размеров данных. Для extendible hashing показано, что операции `get` и `update` остаются самыми быстрыми, тогда как `insert` и особенно `delete` несут дополнительные издержки на модификацию структуры каталогов и бакетов. Для min hash подтверждена линейная масштабируемость операций `build` и `add` (≈ 15 тыс нс/элемент) при квадратичном росте задержки `fullscan` — базового алгоритма перебора, используемого для эталонного сравнения. Результаты представлены в таблицах и на графиках.